МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ПОЛТАВСЬКА ПОЛІТЕХНІКА ІМЕНІ ЮРІЯ КОНДРАТЮКА»

Навчально науковий інститут інформаційних технологій і робототехніки
Кафедра комп'ютерних та інформаційних технологій і систем

Лабораторна робота № 3

з навчальної дисципліни

«Технології розробки корпоративних Web-додатків»

Виконала:

студентка 403-ТН

Кіріяк Ірина Вікторівна

Перевірив:

Мізік Андрій Олегович

Полтава 2026

1. Приклади коду для звіту

Визначення моделі API (DTO)

Використовую **DTO**, щоб відокремити внутрішню структуру бази даних від того, що бачить користувач. Це забезпечує безпеку та гнучкість.

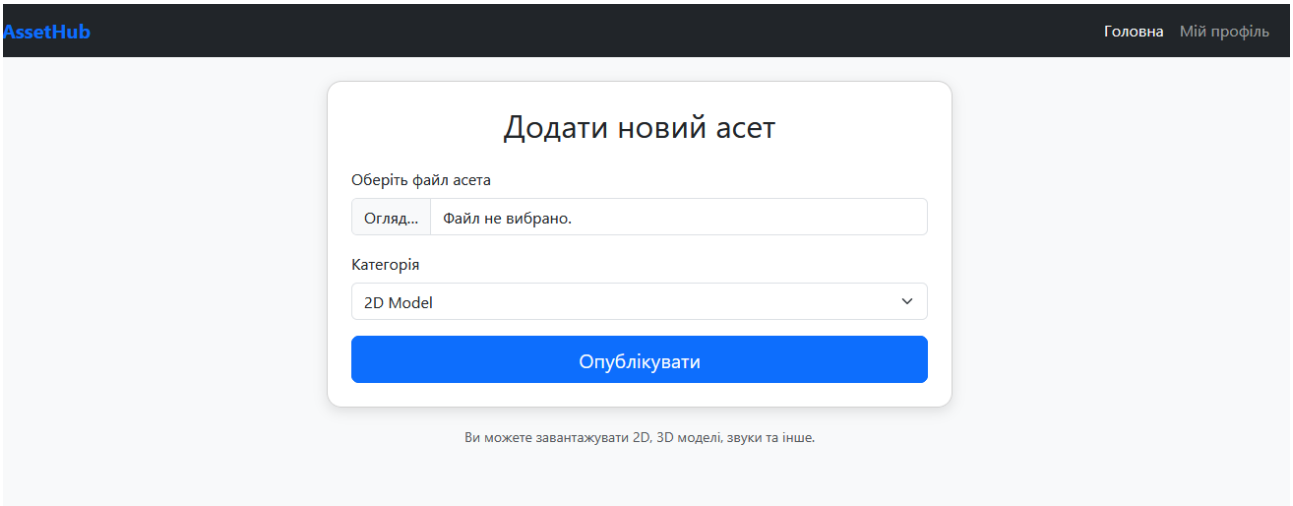
```
    Python
# AssetDTO — об'єкт для передачі даних клієнту
class AssetDTO:
    def __init__(self, asset):
        self.id = asset.id
        self.name = asset.filename
        self.category = asset.category
        # Форматуємо дату для зручного відображення в API
        self.date = asset.upload_date.strftime("%Y-%m-%d
%H:%M")
```

Реалізація методу (Controller Action)

Контролер не працює з базою даних напряму, він делегує це сервісу, що відповідає принципу єдиної відповідальності (Single Responsibility).

```
Python
@app.route('/api/assets', methods=['GET'])
def get_assets():
    # Контролер викликає бізнес-логіку через сервісний шар
    assets_dto = AssetService.get_user_assets(user_id=1)

    # Повертаємо список DTO у форматі JSON зі статус-кодом
200 (OK)
    return jsonify([vars(a) for a in assets_dto]), 200
```



3. Відповіді на контрольні запитання

1. Порівняння REST, GraphQL та gRPC:

- REST: Універсальний, працює через стандартний HTTP. Найкраще підходить для публічних API та простих веб-сайтів (як наш AssetHub).
- GraphQL: Дозволяє клієнту самому обирати, які дані отримати. Ідеальний для мобільних додатків та складних інтерфейсів.
- gRPC: Надшвидкий завдяки бінарному формату. Використовується для внутрішньої взаємодії між мікросервісами, де важлива продуктивність.

2. Over-fetching та Under-fetching:

- Over-fetching: Отримання зайвих даних (наприклад, тягнемо весь профіль користувача, хоча потрібен лише аватар).
- Under-fetching: Отримання замалої кількості даних (треба робити 5 запитів, щоб зібрати інформацію для однієї сторінки).
- GraphQL вирішує це, дозволяючи в одному запиті чітко вказати потрібні поля.

3. Protocol Buffers (Protobuf) vs JSON:

- Protobuf — це бінарний формат (нечитабельний для людини), а JSON — текстовий.
- Переваги Protobuf: Значно менший розмір пакетів, вища швидкість серіалізації та вбудована строга типізація даних.

4. Версіонування API:

- REST: Зазвичай через URL (/api/v1/assets) або заголовки.
- GraphQL: Замість версій використовується еволюція схеми (поступове додавання полів та маркування старих як deprecated).

5. Концепція Idempotency (Ідемпотентність):

- Це властивість операції, при якій повторний запит дає той самий результат, що й перший, не змінюючи стан системи знову.
- Ідемпотентні методи: GET (читання), PUT (повне оновлення), DELETE (видалення). POST — не ідемпотентний, бо кожен запит може створювати новий дублікат запису.

6. Роль SDL (GraphQL) та .proto файлів (gRPC):

- Це "контракти" системи. Вони чітко описують, які типи даних існують та які команди можна виконувати. Це дозволяє розробникам Frontend та Backend працювати незалежно, знаючи структуру даних заздалегідь.